SPRINT 02, QUALIDADE E CI

CI parte 2, SAST e DAST no pipeline

O hook local pega o óbvio. O pipeline enxerga o resto. Análise estática de código, análise dinâmica da aplicação viva.

Aula	06 de 14
Data	13 de maio de 2026, quarta-feira, 10h00 às 12h00
Professor	José Romualdo da Costa Filho
Módulo	13, Engenharia de Software, Turma T13

Agenda da aula

inteli.

Vocês leram os 4 auto-estudos (CI da Atlassian, SAST do GitLab, OWASP Top 10, Semgrep). A aula de hoje pega tudo isso e joga no pipeline de verdade. Três blocos, um hands-on grande, fechamento em 10 minutos.

Bloco	Atividade	Duração
10h00, Daily Time	Check-in da sprint, cronômetro de 15 min	15 min
10h15, Bloco 1	SAST, fundamentos, shift-left, OWASP Top 10	25 min
10h40, Bloco 2	Ferramentas SAST, SonarQube, Semgrep, GitLab SAST, hands-on	50 min
11h30, Bloco 3	DAST como complemento, ZAP, defesa em camadas	20 min
11h50, Fechamento	Quiz, checklist, auto-estudo da Aula 07	10 min

DAILY, 15 MIN

Daily Time

Como saiu o dever de casa da Aula 05? Hook barrou o secret? Imagem buildou? CVE mais sério que o Trivy apontou? Uma volta rápida, uma frase por grupo. Quem ficou travado fala.

15:00

INICIAR PAUSAR ZERAR

Objetivos de aprendizagem

inteli.

Ao fim da aula, cada grupo sai com SAST rodando no pipeline do próprio projeto, pelo menos um achado triado com justificativa e a cabeça calibrada para DAST. Quatro verbos, quatro entregas.

Diferenciar

Separar SAST, DAST e SCA por superfície analisada, momento de execução e tipo de vulnerabilidade detectável.

Configurar

Adicionar uma etapa de SAST ao pipeline do grupo (SonarQube, Semgrep ou GitLab SAST) com critério de severidade que bloqueia merge.

Triar

Ler a saída do scanner, classificar cada achado como fix, aceitar ou suppress, justificando a decisão em uma linha.

Planejar

Posicionar DAST no ciclo e identificar qual tipo de falha ele pega que o SAST deixa passar.

Do laptop para o pipeline, o que muda de verdade

inteli.

Na Aula 05 a gente montou três barreiras no laptop: hook de commit, scanner de secret, build em container. Todas dependem da disciplina do dev. Um `git commit --no-verify` pula tudo. A Aula 06 move essas mesmas validações para um lugar que ninguém consegue pular.

Aula 05, no laptop

- Hook pre-commit com ESLint
- TruffleHog varre o diff
- `docker build` local
- Disciplina do dev como garantia

Aula 06, no pipeline

- SAST como job obrigatório do CI
- Scanner de secret no runner
- Trivy varre a imagem antes do push para registry
- Política de merge exige build verde

Por que duas camadas e não uma

Laptop é rápido, feedback em segundos. Pipeline é confiável, sem bypass. Um não substitui o outro. A barreira local salva o tempo do dev, a do pipeline salva a empresa.

SAST, DAST, SCA, três vistas da mesma aplicação

inteli.

Segurança de aplicação é uma sigla que esconde três técnicas distintas. Cada uma olha para um pedaço diferente e ignora o resto. A matriz abaixo é a bússola do resto da aula.

SAST

Analisa o **código-fonte** sem executar. Acha injection, uso inseguro de API, hardcoded credentials. Cobertura alta, falso positivo alto.

DAST

Ataca a aplicação **rodando**, de fora para dentro, como um cliente. Acha config de runtime, falha de auth, header faltando. Cobertura restrita ao que a app expõe.

SCA

Lê o **manifest de dependências** e casa com base de CVE. Acha lib vulnerável (log4shell, CVE em express). Barata, precisa de atualização constante.

Relação com a Aula 05

Trivy da aula passada é SCA aplicado em imagem de container. Hoje a gente adiciona SAST no pipeline. DAST entra como tema de fechamento. As três camadas, juntas, formam o que o mercado chama de shift-left completo.

BLOCO 1, 25 MIN

SAST, fundamentos e por que shift-left importa

O que o scanner enxerga, o que ele não enxerga, custo por tempo de detecção e o papel do OWASP Top 10.

O que é SAST, na real

inteli.

Static Application Security Testing. O scanner abre seu código, monta uma árvore sintática (AST), aplica um conjunto de regras e devolve uma lista de trechos suspeitos. Tudo sem executar nada. Funciona em cima do texto, parecido com o que o ESLint faz para style, mas com regras focadas em segurança.

Como ele enxerga

Parser monta AST, taint analysis rastreia dado sujo (input do usuário) até um sink perigoso (query SQL, `eval`, exec). Regras descrevem padrões problemáticos.

O que ele acha bem

SQL injection clássico, XSS em string concatenada, hardcoded API key, uso de criptografia fraca (MD5, DES), `eval` com variável, `dangerouslySetInnerHTML`.

O que ele não acha

Falha de config em produção, header de segurança ausente, auth quebrada em runtime, race condition que só aparece com carga. Essas ficam para DAST.

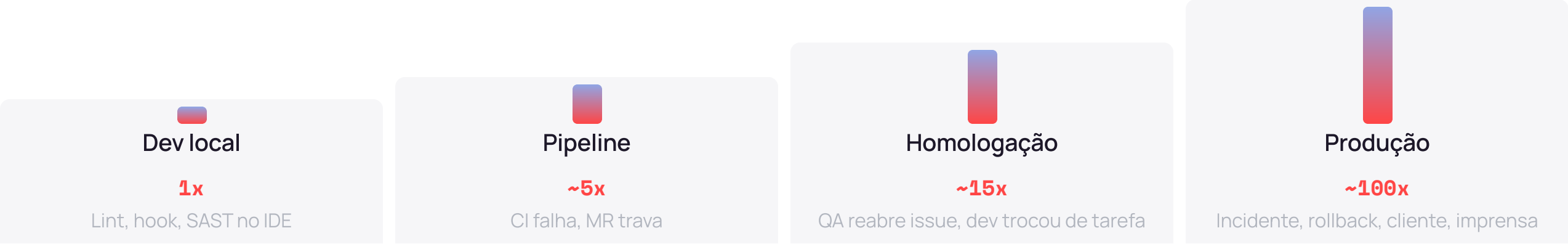
O limite fundamental

SAST analisa o que está escrito, não o que vai acontecer. Se uma variável vem de fonte externa que ele não consegue seguir (banco, env, arquivo), a análise de taint se perde. Por isso falso negativo existe. Ferramenta nenhuma substitui revisão de segurança humana em código crítico.

Shift-left, a matemática do custo

inteli.

A palavra virou jargão e perdeu o sentido. No fundo é só aritmética: quanto mais tarde o defeito é achado, mais caro para corrigir. O gráfico abaixo é uma estimativa clássica do IBM System Sciences Institute, arredondada para a realidade de 2026.



O que isso significa no dia a dia

SAST no pipeline custa 5x mais que no IDE, mas 20x menos que na homologação. Quem pula a camada fica com CVE em produção, e aí a conta explode. Colocar o scanner cedo não é ideologia, é economia de engenharia.

OWASP Top 10, o catálogo que todo scanner carrega

inteli.

OWASP Top 10 é lista consensual das dez classes de vulnerabilidade mais presentes em aplicação web. Edição 2021, ainda em vigor no começo de 2026. Qualquer ferramenta SAST mapeia suas regras para esse catálogo. Três categorias concentram a maioria dos achados reais.

A01:2021

Broken Access Control

Usuário acessa rota que não deveria, manipula ID no path, escala privilégio por flag. Líder do ranking por três edições seguidas.

A03:2021

Injection

SQL injection, command injection, LDAP injection. Dado do usuário concatenado em query. Clássico, continua aparecendo em scanner.

A05:2021

Security Misconfiguration

Header ausente, debug ligado em produção, S3 aberto, senha padrão. SAST pega parte, DAST pega o resto.

Usar como filtro de priorização

Quando o scanner joga 200 achados, um critério pragmático: atacar primeiro A01, A03 e A05. Se sua aplicação tem dado de usuário e formulário de login, essas três classes respondem pela maior parte do risco real. Os outros sete entram na fila, não no bloqueio de merge.

Falsos positivos, a parte que dói

inteli.

Todo SAST gera falso positivo. Regra casou mas o contexto do seu código faz a vulnerabilidade não existir. Ignorar é péssimo, corrigir o que não é problema também. O trabalho real é triar com método, não perseguir zero achado.

Por que aparecem

Regra é genérica, sua aplicação tem proteção que o scanner não enxerga. Ex: você sanitiza input antes da query, mas ele não consegue seguir a função de sanitização.

Como triar

Três rótulos: **fix**, **aceitar**, **suppress**. Fix muda código. Aceitar documenta o risco residual. Suppress marca regra com justificativa no código.

Regra de ouro

Suppress sem justificativa é dívida técnica. Toda supressão tem comentário explicando por que não é real. Revisor do MR lê e bate o olho antes de aprovar.

A armadilha clássica

Turma nova tende a desligar a regra inteira quando dá muito falso positivo. Ruim. A regra detecta algo real, só está mal tunada. Suppress pontual com justificativa é o caminho. Desligar regra deixa porta aberta para o problema de verdade quando ele aparecer.

BLOCO 2, 50 MIN

Ferramentas SAST no pipeline

Quatro opções do mercado, critérios de escolha, hands-on para plugar uma delas no pipeline do grupo.

Quatro ferramentas que dominam o mercado

inteli.

Existem dezenas. No mundo real, quatro concentram a adoção. Diferem em modelo de regra, licença, profundidade de análise e quanto amigam com pipeline. Nenhuma é a melhor em tudo.

MULTI-LINGUAGEM

SonarQube

Server self-hosted, quality gate configurável. Comunidade grande, muita regra. Exige instância rodando.

REGRAS EM YAML

Semgrep

CLI simples, regras legíveis em YAML, registry público com milhares de rules. Roda direto no runner.

GITLAB NATIVO

GitLab SAST

Analyzers prontos por linguagem. Incluído no GitLab Ultimate, resultado aparece no MR. Setup mínimo.

QUERY SEMÂNTICA

CodeQL

Da GitHub, trata código como banco de dados. Query em QL poderosa, uso intenso por security research. Integrado ao GitHub Actions.

Como escolher para a Sprint

Repo no GitLab da Inteli, time começando, linguagem JavaScript/TypeScript: GitLab SAST resolve. Quer controle fino sobre regra, sem server: Semgrep. Precisa de dashboard histórico com métricas: SonarQube. CodeQL brilha em projeto maduro com budget para tuning.

SonarQube, quality gate como contrato

inteli.

SonarQube vai além de SAST puro. Faz também detecção de code smell, duplicação, cobertura. Para segurança, foca em Security Hotspot (ponto a revisar) e Vulnerability (risco confirmado). Quem manda na build é o quality gate, um conjunto de regras que decide se a MR fica "verde" ou "vermelho".

Quality gate típico

- Zero nova vulnerabilidade na MR
- Zero security hotspot não revisado
- Cobertura de novo código maior que 80%
- Duplicação em novo código menor que 3%

Como integra no CI

- Job roda `sonar-scanner` no runner
- Manda resultado para o server
- Server calcula quality gate
- Step seguinte quebra se gate falhar

Pegadinha do gate

Gate padrão olha só código novo, não histórico inteiro. Faz sentido: não dá pra forçar time a arrumar toda dívida técnica de uma vez. Mas aluno tende a achar que gate verde significa projeto impecável. Não significa. Significa que a entrega de hoje não piorou a média.

Semgrep, regra legível como o código

inteli.

O Semgrep parte de uma ideia teimosa: se regra de SAST for mais difícil de escrever do que o código que ela vigia, ninguém vai escrever. Aí a sintaxe virou YAML, o pattern se parece com o trecho procurado, com variáveis no lugar dos nomes. Para grupo que começa, é a menor curva de aprendizado.

```yaml
# semgrep rule, exemplo
rules:
  - id: hardcoded-api-key
    message: API key parece hardcoded
    severity: ERROR
    languages: [javascript, typescript]
    pattern: |
      const $KEY = "..."
    metadata:
      owasp: A05-Security-Misconfiguration
      cwe: CWE-798
```

Como lê na prática

1. **id**: nome da regra no relatório
2. **pattern**: o que procurar no código
3. **$KEY**: variável que casa qualquer nome
4. **metadata**: liga ao OWASP e CWE
5. **severity**: ERROR bloqueia, WARNING avisa

Registry público tem mais de 2000 regras prontas. `semgrep --config auto` detecta a linguagem e aplica o conjunto recomendado.

GitLab SAST, a opção mais barata de começar

inteli.

Vocês estão no GitLab da Inteli. Essa é a ferramenta que sai mais barata de ligar: incluir um template no `.gitlab-ci.yml` e o pipeline ganha um job de SAST. Por baixo do capô, GitLab roda analyzers específicos por linguagem (Semgrep para JavaScript, Bandit para Python, etc).

```yaml
# .gitlab-ci.yml
include:
  - template: Security/SAST.gitlab-ci.yml

sast:
  stage: test
  rules:
    - if: $CI_COMMIT_BRANCH

# Opcional: quebrar pipeline em severidade HIGH
variables:
  SAST_EXCLUDED_ANALYZERS: ""
  SAST_EXCLUDED_PATHS: "spec, test, tests"
```

O que acontece ao merge

1. Pipeline dispara job `sast`
2. Analyzer roda no código da branch
3. Achados viram widget na página do MR
4. Revisor vê severidade e trecho afetado
5. Política de merge pode exigir zero HIGH

Documentação oficial no auto-estudo. Leitura de 30 min que vocês já fizeram. Hoje a gente aplica.

HANDS-ON C, EM GRUPO, 25 MIN

Plugue SAST no pipeline do seu projeto

1. Em grupo, escolham uma ferramenta: GitLab SAST (padrão), Semgrep (CLI) ou SonarQube (se já tem instância). Justifiquem em uma frase.
2. Abram o `.gitlab-ci.yml` do repositório da Sprint e adicionem o job de SAST. Commit direto numa branch `feat/sast-pipeline`.
3. Abram MR para `main` e esperem o pipeline rodar. Tirem print do widget de segurança com os achados.
4. Escolham um achado. Leiam o trecho de código apontado e classifiquem como fix, aceitar ou suppress. Uma linha de justificativa.
5. Se classificaram como fix, mandem o patch no mesmo MR. Se foi aceitar ou suppress, abram issue com a justificativa linkada.

Se a ferramenta não achar nada, o repo está limpo ou o job não rodou. Confiram no log do runner antes de comemorar. Turma do semestre passado levou 10 min para perceber que o include do template estava fora do lugar.

Debrief C, o que sempre aparece na primeira rodada

inteli.

Três padrões se repetem em toda turma que roda SAST pela primeira vez. Mapeio aqui para vocês não gastarem energia redescobrindo o óbvio. Quem pulou o auto-estudo da OWASP vai cair pelo menos em um destes.

1, Enxurrada de HIGH

Primeira rodada vem com 50 achados. Normal. Ferramenta analisa código legado que nunca viu scanner. Triagem começa por A01, A03, A05. Resto entra em fila.

2, Falso positivo em teste

Scanner reclama de `eval` em `tests/`. Exclua a pasta no config. Código de teste tem padrão permissivo por natureza.

3, Merge bloqueado por regra boba

Regra de severity MEDIUM travando pipeline toda hora. Ajuste política: bloqueia em HIGH e CRITICAL, avisa em MEDIUM, ignora LOW.

A pergunta que fica

Se o scanner sempre pega os mesmos 50 achados, ele vira ruído. Time para de olhar. A saída é baseline: marcar o estado atual como aceito e monitorar só o novo. Fica a mesma lógica do SonarQube quality gate em código novo.

Como ler um achado, exemplo real

inteli.

Saída típica do Semgrep num projeto Node da Sprint. Três informações para procurar sempre: trecho de código, regra que disparou e severidade. A sequência é sempre a mesma: entender, decidir, documentar.

```
semgrep scan, academic-tracker
==============================================
src/controllers/student.js:42
  rule: javascript.express.security.injection.tainted-sql-string
  severity: HIGH

    40 | router.get('/search', (req, res) => {
    41 |   const name = req.query.name
  42 |   const sql = `SELECT * FROM students WHERE name = '${name}'`
    43 |   db.query(sql, (err, rows) => res.json(rows))
    44 | })

  ⚠  Input do usuário concatenado em query SQL. Use parameterização.
  OWASP: A03:2021 Injection | CWE-89

Total: 1 (HIGH: 1)
```

Três passos para qualquer achado

1) Ler o trecho, o input mesmo chega sem sanitização? **2) Decidir**, fix com query parametrizada é trivial aqui. **3) Documentar**, commit com mensagem `fix(security): parameterize student search query`, link para OWASP A03 na descrição. Ninguém esquece por que mudou.

BLOCO 3, 20 MIN

DAST, o complemento dinâmico

O que o SAST deixa passar, ferramentas, onde rodar no pipeline e por que não dá para viver só com uma das duas.

O que DAST enxerga que SAST não vê

inteli.

Dynamic Application Security Testing. A ferramenta fica do lado de fora, fala HTTP, manda payload, observa resposta. Não precisa do código-fonte. Por isso ela acha coisa que o SAST é cego: config de runtime, comportamento em produção, diferença entre o que o código diz e o que o servidor realmente responde.

Como funciona

Crawler descobre endpoints, fuzzer testa payload malicioso em cada um, analyzer interpreta respostas. Assinatura de vuln vem do mesmo OWASP Top 10.

O que acha bem

Header de segurança faltando (CSP, HSTS), cookie sem Secure, CORS aberto, endpoint exposto sem auth, redirecionamento aberto, XSS refletido.

O que não acha

Lógica de negócio (escalar privilégio por regra quebrada), SQL injection em endpoint que o crawler não descobriu, criptografia fraca em dado em repouso.

Trade-off crítico

DAST precisa da aplicação rodando. Isso significa ambiente de preview, seed de dados, tempo de scan (minutos, às vezes hora). SAST roda em segundos sem nada disso. Por isso SAST fica em toda MR, DAST entra em job noturno ou em ambiente de homologação.

Três DAST que o mercado usa

inteli.

Menos fragmentado que SAST. Três nomes resolvem 95% dos casos. Cada um com um papel um pouco diferente na cadeia.

OWASP ZAP

Open source, mantido pela OWASP. Roda CLI em pipeline via `zap-baseline.py`. Relatório HTML. Escolha default para quem começa e não quer licença.

Burp Suite

Proxy interativo com versão paga robusta. Favorito de pentester. Para pipeline tem Burp Enterprise. Caro, mas profundo em achado de lógica.

StackHawk

Filha do ZAP, feita para CI. Integra com GitHub Actions, pipeline de config simples, foco em app moderna com API. Modelo SaaS.

Para a Sprint do módulo

ZAP é a escolha óbvia: gratuito, roda em container, relatório fácil de anexar em MR. Não vamos aplicar hoje, mas fica como conhecimento para quando a equipe tiver ambiente de preview no projeto da Sprint 03.

Onde plugar DAST, sem travar o fluxo do time

inteli.

O erro comum é colar DAST na MR como se fosse SAST. Aí o pipeline demora meia hora, o time para de rodar o scan, o valor evapora. Três posições fazem sentido na vida real.

1, Preview env por MR

Cada MR sobe ambiente efêmero. ZAP baseline roda em 3 min, pega só o óbvio (header, cookie, endpoint sem auth). Feedback rápido, não bloqueia merge.

2, Scan noturno em staging

ZAP full scan roda contra staging toda noite. Demora 30 min a 2 h, relatório vai para o time de segurança. Pega o que baseline ignora.

3, Scan antes de release

Pré-deploy de produção, DAST contra candidato. Gate manual, time lê achado, decide se sobe. Não automatiza, não deveria.

Regra de maturidade

Time novo começa só com posição 1 ou 2. Posição 3 exige alguém que saiba ler saída de DAST e separar ruído de ameaça. Tentar ter os três antes da hora gera dashboard que ninguém lê, que é pior que não ter dashboard nenhum.

Defesa em profundidade, SAST, DAST, SCA juntos

inteli.

Nenhuma ferramenta pega tudo. A ideia é sobrepor vistas: uma olha código, outra olha dependência, outra olha comportamento. A matriz a seguir é o retrato de um pipeline de CI maduro, o que a Sprint 03 deveria ter como alvo.

SAST	Roda em cada MR. Analisa código novo, bloqueia merge em severity HIGH. Feedback em minutos. Tempo de scan: 2 a 5 min.
SCA	Roda em cada MR e em schedule. Analisa dependências (Trivy em imagem, Dependabot no `package.json`). Alerta em CVE CRITICAL, abre MR automática de atualização.
DAST	Roda em preview env da MR (baseline 3 min) e em staging toda noite (full scan). Não bloqueia merge, gera issue priorizada. Exige app rodando.

O cenário ideal em uma frase

SAST impede código ruim de entrar. SCA impede dependência ruim de ficar. DAST impede config ruim de sair. A sobreposição das três forma o que se chama de defesa em profundidade. Tirar qualquer uma deixa um ponto cego proporcional.

FECHAMENTO, 10 MIN

Checagem, checklist, próxima aula

Duas perguntas, uma checklist de cinco linhas, três provocações para a Aula 07.

Checagem 1 de 2, SAST

inteli.

Seu SAST apontou 50 achados numa base que nunca passou por scanner antes. Qual é a primeira ação mais sensata?

- Desligar todas as regras que geraram falso positivo, reduzir ruído
- Estabelecer baseline com o estado atual e configurar o gate para monitorar só código novo
- Bloquear todos os merges até que os 50 sejam corrigidos
- Apagar as regras de severity HIGH, começar pelas LOW

Checagem 2 de 2, SAST vs DAST

inteli.

Sua aplicação esqueceu o header `Content-Security-Policy` em produção. Qual ferramenta tem a melhor chance de detectar isso?

- SAST, porque analisa o código que configura o servidor
- DAST, porque fala HTTP com a aplicação viva e inspeciona cada resposta
- SCA, porque cheta as dependências de rede
- Scanner de secrets, porque CSP é uma string sensível

Checklist de segurança no pipeline, para a Sprint

inteli.

Cinco linhas que um pipeline de segurança minimamente decente tem. Não precisa implementar tudo hoje. Precisa saber qual está cobrindo e qual ficou de fora, com justificativa.

1, Hook
pre-commit com lint e secret scanner local.

2, SAST
Job no pipeline, severidade HIGH bloqueia merge.

3, Secret
Segundo scanner no runner, TruffleHog ou Gitleaks.

4, SCA
Trivy em imagem, CVE CRITICAL quebra build.

5, DAST
ZAP baseline em preview env, scheduled full scan.

A regra do pipeline verde

"Build verde" em 2026 não é teste passando. É código sem vuln HIGH, sem secret no diff, sem CVE CRITICAL na imagem, sem header faltando em preview. Quem ainda trata build como "rodou o `npm test`" está fazendo CI de 2015. Sprint 03 vocês vão sentir essa mudança de definição na prática.

Auto-estudo para a Aula 07

inteli.

A Aula 07 fecha a Sprint 02 com o tema de Continuous Deployment. Pipeline verde, imagem limpa, já viram. Próxima pergunta: e o deploy em si? Três provocações para chegar pronto.

Leitura, Continuous Delivery vs Continuous Deployment

Post curto da Atlassian, "Continuous integration vs delivery vs deployment". 10 min. Fique com a diferença entre as três palavras.

Leitura, Deployment strategies

AWS Builders Library, "Achieving Rapid Safe Deployment". Foco em blue/green e canary. 20 min, vale cada minuto.

Vídeo, GitHub Actions deploy

Tutorial prático, 12 min, deploy automático de app Node pro Fly.io ou Vercel usando GitHub Actions. Serve de referência para o Hello World da Aula 07.

Entrega desta aula, para o AdaLove

Um print do widget de segurança da MR com o SAST rodando. Uma linha dizendo qual foi o achado mais sério que o grupo triou e qual decisão tomou (fix, aceitar, suppress). Uma frase respondendo: se tivesse que incluir DAST na próxima sprint, onde plugaria no pipeline?

inteli.

Até a próxima

Aula 07, Hello World, Continuous Deployment, 19/05